PAGE 1 — LLD CORE (SOLID + PATTERNS)

🔹 **LEFT COLUMN — SOLID + DESIGN BASICS**

SOLID Principles

- **SRP** – One reason to change
- **OCP** – Open for extension, closed for modification
- **LSP** – Subtypes replace base without breaking behavior
- **ISP** – No fat interfaces
- **DIP** – Depend on abstractions, not implementations

Design Philosophy

- **Composition > Inheritance**
- Prefer *Has-a* over *Is-a*
- Loosely coupled, highly cohesive classes

🔹 **RIGHT COLUMN — DESIGN PATTERNS**

Creational

- **Singleton** – Single instance (Logger, Config)
  ⚠ Hard to test, prefer Enum
- **Factory** – Object creation via abstraction
- **Builder** – Step-by-step object creation

Structural

- **Adapter** – Make incompatible APIs work together
- **Decorator** – Add behavior dynamically
- **Facade** – Simplified interface to complex subsystems

Behavioral

- **Observer** – Event subscription model
- **Strategy** – Interchangeable algorithms
- **Command** – Encapsulate requests (Undo/Redo)

📄 **PAGE 2 — HLD + LLD IMPLEMENTATION**

Goal

👉 *What to build & how components interact*

1. Requirements

- Functional: APIs, flows

- NFRs: Scalability, Availability, Latency, Security, Observability

2. System Components

- Client (Web/Mobile)
- API Gateway
- Microservices
- DB (SQL / NoSQL)
- Cache (Redis)
- MQ (Kafka / RabbitMQ)
- Object Store (S3)
- Auth (JWT / OAuth)

3. Architecture Patterns

- Monolith → Microservices
- Layered
- Event-Driven
- CQRS
- Saga

4. Data Design

- DB choice & justification
- Read vs Write
- Indexing
- Sharding
- Caching

🔹 **RIGHT COLUMN — LLD IMPLEMENTATION**

5. Scalability

- Horizontal scaling
- Load balancer
- Stateless services
- Auto-scaling (HPA)
- Cache-aside

6. Availability & Reliability

- Replication

- Failover
- Circuit breaker
- Retry + Backoff
- Idempotency

7. Security

- TLS everywhere
- JWT validation
- Roles / Scopes
- mTLS
- Rate limiting
- Secrets (Vault / KMS)

8. Observability

- Structured logs
- Metrics
- Distributed tracing
- Correlation IDs
- Alerts

9. LLD Core

- Class design (SRP, DI)
- OOP principles
- Thread safety
- API design
- Persistence
- Caching
- Exception handling
- Testing